ASSIGNMENT 5 - CO5

PART A - DESIGN

TASK A1 - EVALUATE THE LINUX SYSTEM COMPONENTS

7 Marks | Bloom's Level: K5 - Evaluate

Report body follows the question paper's A1.1, A1.2, A1.3 and alternative-concept structure.

ASSIGNMENT 5 - CO5

PART A - DESIGN | TASK A1 - EVALUATE THE LINUX SYSTEM COMPONENTS

7 Marks | Bloom's Level: K5 - Evaluate

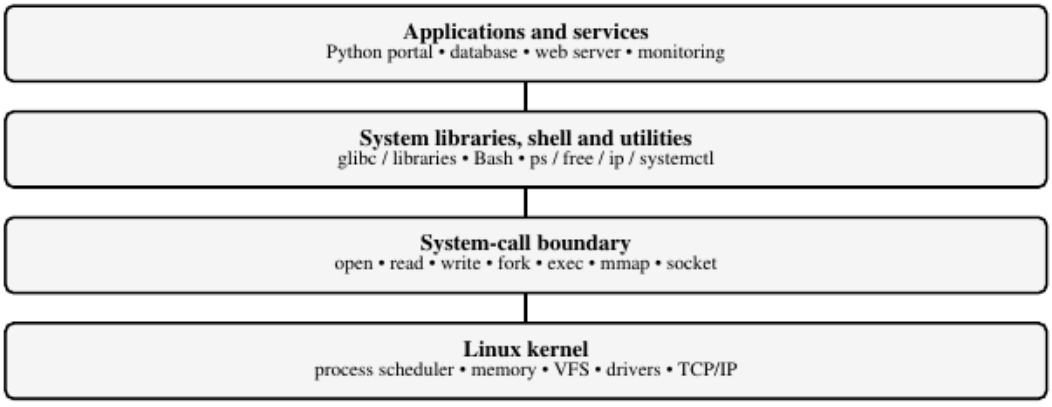
Case study: University Result Portal

The assignment describes a university result portal implemented as a Python web application with a database. For most of the year it serves a few hundred users per day, but 150,000 students log in within two hours on result day. The portal currently runs on one physical Linux server that crashes during the peak. The university has a small private server room and an account with a public cloud provider.

A simple average over the two-hour window is 150,000 / 7,200 = 20.83 logins per second. This average does not describe short bursts and does not equal HTTP request rate: each login can generate multiple application, static-content and database requests. The design must therefore monitor actual request rate, CPU run queues, memory pressure, storage and network I/O rather than size the server from the login average alone.

1. A1.1 - Linux Architecture (2 Marks)

1.1 Main Linux components



Component	Role and relationship to the kernel
Linux kernel	Privileged core that manages CPU scheduling, process/thread state, virtual memory, system calls, file systems, device drivers, interrupts, networking and security enforcement.
System libraries	User-space libraries such as glibc provide reusable APIs and wrappers around system calls, so applications do not need to implement low-level kernel interfaces themselves.
Shell	A command interpreter such as Bash. It launches programs, connects pipelines, expands variables and supports scripts; it is a user-space program, not the kernel.
Utilities	User-space tools such as ps, top, free, vmstat, df, ip and systemctl. They call libraries and system calls to inspect or manage the system.

The separation is important: the shell and utilities request services, while the kernel performs privileged operations. A command such as `cat file.txt` uses a library/system-call path to ask the kernel to open and read the file; the kernel's VFS dispatches the request to the mounted filesystem and storage driver.

1.2 How the kernel manages resources

Kernel area	What it manages	Relevance to the result portal
Processes and CPU	Creates and schedules tasks/threads, maintains process states and identifiers, switches execution contexts, handles signals and enforces CPU scheduling policy.	Web workers, database processes and background tasks compete for CPU. Run-queue growth and CPU saturation increase response time.

Kernel area	What it manages	Relevance to the result portal
Memory	Provides virtual address spaces, page tables, demand paging, memory allocation, reclaim and swap management.	The Python workers and database need working-set memory. Memory pressure can trigger reclaim, swapping and eventually OOM handling.
Files	VFS resolves pathnames and provides common open/read/write/stat interfaces; filesystem-specific code handles on-disk structures and allocation.	The portal uses configuration, application files, logs, database files and cached assets through one kernel file interface.
Devices	Drivers control storage, network and other devices; interrupts and DMA support efficient I/O completion and data movement.	Disk latency, device queues and NIC performance affect database queries and web responses.
Network	Implements sockets and protocol processing, routing interfaces and packet handling; NIC drivers transfer packets to/from hardware.	TCP connections, listen queues, retransmissions and bandwidth determine whether result-day clients can reach the service.

Evaluation: Linux provides a unified, modular resource-management layer. The benefit is that application processes use standard interfaces, and administrators can observe or constrain resource use. The limitation is that a single physical server still has finite CPU, RAM, storage and network capacity. Kernel scheduling cannot compensate for an application/database architecture that receives more concurrent work than the machine can serve. For the case study, split monitoring by resource, identify the bottleneck and scale stateless application workers across more than one host; retain a separately protected database tier.

2. A1.2 - Process and Memory Management in Linux (3 Marks)

2.1 Completely Fair Scheduler (CFS)

The classic Completely Fair Scheduler aims to share CPU time fairly among runnable normal tasks. It models an ideal processor that runs all tasks proportionally, and approximates that model on real hardware by tracking each scheduling entity's virtual runtime (`vruntime`). In the classic CFS design, runnable entities are ordered by virtual runtime and the task owed the most CPU time is selected. Nice values change a task's weight and therefore its proportional share.

Strength	Limitation / evaluation
Prevents one ordinary CPU-bound process from permanently owning the CPU; helps distribute CPU time among runnable tasks of comparable priority.	Fair CPU sharing is not a latency guarantee. If too many requests arrive, every worker may receive a smaller share and queueing still grows.
Supports prioritisation through nice weights and works for general-purpose interactive/server workloads.	It does not solve database locks, slow queries, network bottlenecks or poor thread-pool sizing.
Provides a reasonable default policy for mixed user and service processes.	CPU quotas/cgroups, worker counts, affinity and real-time policies may be relevant for specialised workloads, but should be configured from measurements.

Version qualification: the assignment explicitly asks for CFS, so the classic virtual-runtime model is evaluated here. Current Linux kernel documentation states that the kernel began transitioning the fair scheduler toward EEVDF in Linux 6.6. EEVDF continues the fair-share goal but selects eligible tasks using virtual deadlines, which can improve responsiveness for latency-sensitive tasks. Record `uname -r` and describe the scheduler in the actual Linux kernel being inspected; do not state that every current Linux kernel uses only the original CFS selection algorithm.

For the result portal, fairness is useful because Python workers, database processes and system services all need CPU time. However, the server failure is likely a capacity/architecture problem as much as a scheduler problem. Monitor CPU utilisation, load average, per-process CPU, and pressure; then add application replicas or capacity rather than relying only on scheduler tuning.

2.2 Page cache and swapping

The Linux page cache keeps recently read file data in RAM so repeated reads can be served without another storage access. Buffered writes first update cached pages; dirty pages are written back to storage later. This improves throughput and reduces repeated disk I/O. A large `buff/cache` value in `free -h` is not automatically a memory leak: clean cached pages can be reclaimed when applications need RAM.

Swap provides backing storage for selected anonymous memory pages when RAM is under pressure. It can help the system retain less-active memory pages, but disk/SSD access is much slower than RAM. If a database or web server's active working set is repeatedly swapped in and out, latency can increase sharply (thrashing). A system with no swap can reach memory exhaustion sooner; a system with excessive swap activity may remain alive but deliver unacceptable response times. The decision is workload-specific.

Observation	Command / interface	How to interpret it
Available RAM and cache	`free -h`, `/proc/meminfo`	Check MemAvailable, Cached, Buffers, SwapTotal and SwapFree rather than interpreting used RAM alone.
Swap activity	`swapon --show`, `vmstat 1 5`	In vmstat, repeated non-zero si/so values during load indicate swap-in/swap-out activity.
Memory pressure	`/proc/pressure/memory`	Some/full stall time indicates tasks are delayed because memory resources are constrained.
Per-process memory	`/proc/<PID>/status`, `ps`	RSS, VmSize and VmSwap help identify processes consuming or swapping memory; RSS is resident memory, not total virtual allocation.

For the portal, reserve sufficient RAM for the database buffer/cache and application workers, set realistic worker counts, and monitor `MemAvailable`, swap-in/out and memory pressure during a load test. The page cache should be considered as useful reclaimable memory, but database and worker working sets must not be allowed to force continuous swapping.

2.3 `/proc` and `/sys` as observation tools

Interface	What it exposes	Useful A1 observations
/proc	A pseudo-filesystem exposing kernel and process information. It includes one PID directory per process and global files such as meminfo, stat, loadavg, swaps and filesystems.	`/proc/<PID>/status` for process state and memory; `/proc/meminfo` for system memory; `/proc/stat` for CPU counters; `/proc/loadavg` for load average.
/sys	A RAM-backed interface to kernel objects and their attributes. Its hierarchy mirrors device, bus, class, driver, block and power relationships.	`/sys/class/net` for network interfaces; `/sys/block` for block devices and queue attributes; `/sys/devices` and `/sys/bus` for device/driver relationships.

These are observation interfaces, not ordinary data directories. `/proc` is useful for live process/kernel state; `/sys` is useful for device and driver topology and selected runtime attributes. Some files are read-only, while some attributes accept privileged writes that can change kernel/device behaviour. For an assignment evidence capture, use read-only commands and do not write arbitrary values to `/proc/sys` or `/sys.

Evaluation: `/proc` and `/sys` expose detailed state without requiring a separate monitoring agent, which is useful for diagnosing the result-day failure. Their values are live snapshots and must be interpreted over time. For example, one `vmstat` sample is weaker evidence than a short series taken while a controlled load is running; `/sys` attributes can also be absent in virtualised or WSL environments.

3. A1.3 - Linux File System and Permissions (2 Marks)

3.1 Linux directory hierarchy

Directory	Typical purpose
/	Root of the single directory tree; all mounted filesystems appear within this namespace.
/bin and /sbin	Essential user commands and system/administration utilities (often merged into /usr on modern distributions).
/boot	Bootloader files and kernel/initramfs images.
/dev	Device nodes and interfaces representing devices.
/etc	Host-specific system and service configuration.
/home and /root	Home directories for ordinary users and the root account.
/lib and /usr/lib	Essential and general shared libraries, modules and program support files.
/usr	Most user commands, libraries, shared data and installed software; generally static program content.
/var	Variable data such as logs, caches, state, spool and databases.
/tmp	Temporary files.
/proc and /sys	Pseudo-filesystem interfaces for process/kernel data and kernel object/device attributes.
/srv, /opt, /mnt and /media	Service data; add-on software; temporary mount point; removable media mount point.

The hierarchy separates configuration (`/etc`), installed software (`/usr`), changing operational data (`/var`) and user data (`/home`). For the result portal, a sensible layout keeps service configuration under `/etc`, logs under `/var/log`, database state under a dedicated `/var/lib` or mounted data volume, and service-provided content under `/srv` where appropriate. Separating logs and database files from the root filesystem can make capacity monitoring and recovery easier.

3.2 Virtual File System (VFS)

The VFS is a kernel abstraction between user-space file operations and concrete filesystem implementations. Applications call common system calls such as `open()`, `read()`, `write()`, `stat()` and `chmod()`. VFS performs pathname lookup and uses objects such as dentries, inodes, file objects and superblocks before dispatching operations to the mounted filesystem. This lets ext4, XFS, tmpfs, procfs, sysfs and network filesystems coexist behind one interface.

Evaluation: VFS simplifies application development because the Python portal does not need to know the internal format of every filesystem. The dentry and inode caches speed up repeated path and metadata lookups. However, performance still depends on the backing filesystem, storage device, mount options and cache state. A slow disk or overloaded network filesystem can still delay file operations.

3.3 Linux permissions: owner, group and others

Traditional Unix permissions are stored for three classes: owner (user), group and others. Each class can have read (`r`), write (`w`) and execute (`x`) permissions. For regular files, read opens file content, write modifies content, and execute allows execution. For directories, read lists names, write changes directory entries, and execute/search allows traversal or lookup of entries.

Example	Meaning
`640` (`rw-r-----`)	Owner can read/write; group can read; others have no permissions. Suitable for a private configuration/secret file when group read is deliberately required.
`750` (`rwxr-x---`)	Owner can list/read/write/traverse; group can read/traverse; others have no access. Suitable for a service directory shared with an authorised group.
`700` (`rwx------`)	Only the owner can access the directory. Useful for private service credentials or user data.

The least-privilege approach is to run the portal under a dedicated non-root service account, assign only the necessary group access to application files, make secrets readable only by the service/admin, and avoid world-writable application directories. Permission checks protect files even if a user discovers a path; access should not depend on obscurity.

3.4 setuid and sudo

The set-user-ID (setuid) permission bit on an executable causes it to run with the effective user ID of the file owner under applicable filesystem/kernel rules. It is a specialised privilege mechanism and can create a serious security risk if the program is vulnerable; custom setuid programs should be avoided unless there is a compelling, reviewed need. The bit is commonly shown as an `s` in the owner's execute position (for example, mode `4755`), subject to mount and kernel restrictions.

`sudo` is a policy-controlled tool that lets an authorised user run a permitted command as root or another user. The policy is typically configured in `/etc/sudoers` and should be edited with `visudo`. Unlike granting every administrator a permanent root shell, sudo can restrict which users run which commands and provides an auditable elevation path. It should follow least privilege and protect the sudoers configuration.

For the university portal, the application process should not run as root. Administrative tasks such as package updates, firewall changes and service configuration should be performed through controlled sudo privileges. The database account and portal service account should have separate ownership and permissions.

4. Alternative Concept - Windows Server Instead of Linux

A Windows Server alternative is technically possible: install Windows Server 2025, host the Python application as a managed service behind IIS or a reverse proxy, deploy the supported database engine, and use PowerShell/Desired State Configuration (DSC), Windows Admin Center and Group Policy for administration. Multiple application instances and database high availability would still be required. Changing the operating system alone does not remove the 150,000-user peak or the single-server failure mode.

Measure	Ubuntu Server 24.04 LTS reference	Windows Server 2025 reference
Minimum RAM to install	1.5 GB for ISO installation; Canonical suggests 3 GB or more.	2 GB for Server Core; 2 GB minimum for Desktop Experience and 4 GB recommended.
Minimum system disk	5 GB for ISO installation; Canonical suggests 25 GB or more for a useful installation.	32 GB system partition is stated as an absolute minimum; Server Core is about 4 GB smaller than Desktop Experience.
What the numbers mean	Published system requirements, not actual idle/runtime consumption for the portal.	Published system requirements, not actual idle/runtime consumption for the portal.

The minimum figures suggest that the Windows Server installation baseline needs a larger system partition than the Ubuntu Server 24.04 minimum. Memory minima are closer; the Windows Server Desktop Experience recommendation is 4 GB. These are minimum installation requirements, not a fair benchmark of post-update memory usage or production footprint. Actual use depends on selected roles, GUI, security tools, updates, drivers, page files, logs and monitoring agents.

4.1 Time to automate administration tasks

Windows can automate repeated setup through PowerShell scripts, DSC configurations, Windows Admin Center and Group Policy. Linux can automate the same class of work through shell scripts, systemd and configuration tools such as Ansible. Both approaches require initial design/testing, but reduce repeated manual steps and configuration drift.

Illustrative ten-server rollout	Manual approach	PowerShell/DSC automated approach
Task set	Create a service account; install runtime/packages; configure firewall; deploy service configuration; enable logging/monitoring.	Same task set expressed as a version-controlled PowerShell/DSC configuration and applied remotely.
Estimated time assumption	15 minutes per server × 10 = 150 minutes.	45 minutes to create/test the configuration + 2 minutes per server to apply/verify = 65 minutes for the first rollout.
Repeat rollout	Another 150 minutes if repeated manually.	2 minutes per server × 10 = 20 minutes if the configuration is already tested.

Under these assumptions, the first automated rollout saves 85 minutes (56.7%) compared with the manual baseline; later repeat runs save more because the 45-minute authoring cost is not repeated. This is an illustrative planning calculation, not a measured result from a Windows Server machine. Actual time depends on server count, network reachability, security policy, package availability, testing and exception handling. The same measurement method should be used to compare PowerShell/DSC and Linux automation in the university's own environment.

4.2 Windows Server licence cost per server

Microsoft's Windows Server 2025 US pricing page lists a suggested MSRP of USD 1,176 for Standard and USD 6,771 for Datacenter, with pricing based on 16-core licences. Microsoft states that prices vary by country and directs customers to a reseller for a quote. The India pricing page currently displays the MSRP as variable. The listed price is therefore a US reference, not an India quotation.

Edition	US suggested MSRP (16-core licence)	Typical fit
Windows Server 2025 Standard	USD 1,176	Physical or minimally virtualised environment.
Windows Server 2025 Datacenter	USD 6,771	Highly virtualised datacentres and cloud environments.

The above prices exclude the full project cost. Microsoft states that Windows Server CALs are required for each user or device accessing a server under the applicable licensing model. Core-count minimums, virtualisation rights, CAL type/count, Software Assurance, reseller terms and the university's user-access model must be confirmed before purchase. With 150,000 student users, CAL or other access licensing can materially affect the total and should not be ignored.

Linux Server distributions such as Ubuntu can be installed without a per-server Windows Server licence fee; optional enterprise support, management and extended-security subscriptions may add cost. The economic comparison should include support, staff skills, third-party software, migration and operations—not only the operating-system licence.

4.3 Evaluation for the result portal

Windows Server offers familiar enterprise administration tools and PowerShell automation, but its minimum system-partition requirement and core/CAL licensing can make the base platform more expensive for a large internal user population. Linux offers a smaller minimum install footprint in the Ubuntu Server reference and avoids a Windows Server per-core licence, while providing the kernel interfaces and tooling already used by the existing portal. However, Linux also needs competent administration, security updates, backups and monitoring.

Recommendation: retain Linux as the portal's operating-system platform unless a specific Windows dependency or institutional support requirement justifies migration. The immediate corrective action is architectural: remove the single-server bottleneck by separating the stateless Python application from the database, adding a load balancer and multiple application instances, monitoring resource pressure, tuning worker/connection pools from measured load, and ensuring database backup/failover. CFS, page cache, swap, `/proc`, `/sys`, VFS and permissions are

operational tools that support that redesign; they are not substitutes for capacity planning.

5. Evidence Collection Procedure

A1 does not ask for a C program or a Complete-the-Code exercise. The included Bash script is an optional, read-only evidence collector for the Linux concepts assessed in A1. Run it on your own Ubuntu/WSL environment so the evidence describes your machine rather than the report-preparation environment.

```
chmod +x a1_linux_evidence.sh
./a1_linux_evidence.sh

# Optional: choose an output filename
./a1_linux_evidence.sh A1_my_system_evidence.txt
```

The resulting text file includes kernel/distribution details, CPU/process observations, memory/swap/pressure data, selected `/proc` entries, `/sys` network/block information, mounted filesystems and permission examples. Commands that are not installed or attributes unavailable in WSL are noted rather than treated as failures. Keep the generated text output and a terminal screenshot as your local evidence.

6. Conclusion

Linux is composed of a privileged kernel and user-space libraries, shells, utilities and applications. Its process scheduler distributes CPU time, memory management combines virtual memory, page cache and swap, VFS abstracts filesystem implementations, and drivers/network stacks mediate hardware I/O. `/proc` and `/sys` expose live state for diagnosis. The directory hierarchy and owner/group/others permissions support organised, least-privilege administration. Windows Server can host the portal and offers PowerShell/DSC automation, but its installation footprint and per-core/CAL licensing should be weighed against Linux. Most importantly, neither OS can make a single physical server support an unbounded result-day spike: the portal must be scaled and made resilient.

7. Technical References

[1] Linux Kernel Documentation, *CFS Scheduler* - classic fair scheduling and virtual runtime. https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html

[2] Linux Kernel Documentation, *EEVDF Scheduler* - transition from CFS beginning in Linux 6.6 and fair scheduling with virtual deadlines. https://docs.kernel.org/scheduler/sched-eevdf.html

[3] Linux Kernel Documentation, *Memory Management Concepts* - virtual memory, page cache, anonymous memory and reclaim. https://docs.kernel.org/admin-guide/mm/concepts.html

[4] Linux Kernel Documentation, */proc Filesystem* - process-specific and system information. https://www.kernel.org/doc/html/latest/filesystems/proc.html

[5] Linux Kernel Documentation, *sysfs* - exported kernel objects and device hierarchy. https://docs.kernel.org/filesystems/sysfs.html

[6] Linux Kernel Documentation, *Virtual File System Overview* - common file interface, dentry/inode/file objects. https://docs.kernel.org/filesystems/vfs.html

[7] FreeDesktop / Linux Foundation, *Filesystem Hierarchy Standard 3.0* - Linux directory placement. https://specifications.freedesktop.org/fhs/latest/

[8] Canonical, *Ubuntu Server 24.04 LTS System Requirements* - minimum and suggested memory/storage. https://ubuntu.com/server/docs/system-requirements/

[9] Microsoft Learn, *Windows Server 2025 Hardware Requirements* - minimum RAM and system-partition storage. https://learn.microsoft.com/en-us/windows-server/get-started/hardware-requirements?pivots=windows-server-2025

[10] Microsoft, *Windows Server 2025 Pricing and Licensing* - US MSRP, 16-core price basis and CAL requirement. https://www.microsoft.com/en-us/windows-server/pricing ; India page: https://www.microsoft.com/en-in/windows-server/pricing

[11] Microsoft Learn, *PowerShell Desired State Configuration* and *Windows Server Management* - repeatable configuration and administration tools. https://learn.microsoft.com/en-us/powershell/dsc/getting-started/winGettingStarted ; https://learn.microsoft.com/en-us/windows-server/administration/manage-windows-server

[12] Linux man-pages, *chmod(1)* and *sudo(8)* - special permission bits and policy-controlled privilege elevation. https://man7.org/linux/man-pages/man1/chmod.1.html ; https://man7.org/linux/man-pages/man8/sudo.8.html

Source alignment: Assignment 5 - CO5, Part A - Design, Task A1 - Evaluate the Linux System Components, 7 Marks, page 43 of the supplied question paper. General submission requirements appear on page 42.